Extending Heterogeneity-Aware GPU Scheduling with Fragmentation Awareness

Varun Ramesh
Stanford University
Graduate Department of Engineering
vramesh3@stanford.edu

Jiayu Chang
Stanford University
Graduate Department of Engineering
cjy1125@stanford.edu

Hyeonggyu Kim
Stanford University
Graduate Department of Engineering
hgk@stanford.edu

Stefan Gabriel Ene
Stanford University
Graduate Department of Engineering
enegstef@stanford.edu

Abstract—We integrate Fragmentation Gradient Descent (FGD) into Gavel's heterogeneity-aware scheduling framework through a two-stage design: Gavel's policy layer determines GPU type and quantity allocations via linear programming, while FGD selects the specific nodes that minimize fragmentation growth. We replicate both systems independently – validating Gavel on the Microsoft Philly trace (504 experiments) and FGD on the Alibaba GPU-sharing trace (6 placement policies across 1,200 nodes) – then evaluate the combined scheduler on two Alibaba cluster topologies under both Max-Min Fairness and FIFO policies (900 total experiments). Our results reveal that fragmentation-aware placement reduces the fragmentation rate by 30–40% on clusters with mixed node sizes (1–8 GPUs per node), but provides no benefit on uniform-node clusters where all placement strategies converge. This finding demonstrates that the value of fragmentation-aware scheduling depends on node topology diversity, not GPU type heterogeneity, and that effective cluster scheduling requires jointly considering allocation policy, placement strategy, and physical topology.

Index Terms—GPU sharing, fragmentation, heterogeneity-aware, scheduling policies, workload traces, distributed training

I. Introduction

GPU cluster scheduling for deep learning has evolved as organizations deploy increasingly diverse hardware for training and inference workloads. In production environments, GPU utilization averages between 25% and 50% [2], motivating the adoption of GPU sharing where multiple tasks run on a single physical GPU through partial resource allocation.

While GPU sharing improves aggregate utilization, it introduces resource fragmentation: remaining capacity on individual devices may be too small to satisfy subsequent requests, and these fragments cannot be consolidated across physical GPU boundaries. Production trace analysis from Alibaba clusters shows that fragmentation renders 21% to 42% of unallocated GPU resources unusable during normal operation [2], [3].

Separately, heterogeneous GPU clusters require scheduling policies that account for varying per-model speedups across accelerator types. A ResNet-50 model sees 10x throughput improvement from K80 to V100, while A3C sees only 2x [1].

Two recent systems address complementary aspects of this problem. Gavel [1] expresses scheduling policies as optimization problems over an effective throughput abstraction, enabling heterogeneity-aware allocation with support for fairness, makespan minimization, and hierarchical policies – but was validated exclusively on whole-GPU workloads where fragmentation does not arise. FGD [2] defines a statistical fragmentation measure and places tasks toward the steepest descent of this measure, reducing unallocated GPUs by up to 49% – but operates under FIFO ordering without a flexible policy framework.

A. Contribution

We bridge these approaches through three contributions:

1) Independent replication of both Gavel and FGD, validating against published results on their respective traces.
2) Integration of FGD placement into Gavel's two-stage architecture, preserving policy-level optimization while adding fragmentation awareness.
3) A comparative evaluation revealing that fragmentation reduction depends on node topology diversity (mixed node sizes), not GPU type heterogeneity – a finding neither original paper addresses.

II. Background & Related Work

A. Heterogeneity-Aware Scheduling with Gavel

Modern GPU clusters contain multiple accelerator generations with widely varying per-model throughputs. Gavel [1] addresses this by expressing scheduling policies as optimization problems over allocation matrices, where

*This paper was written as a final project report for Stanford's Advanced Networking and Distributed System CS244 course in Winter 2026. Thanks to Dr. Keith Winstein and Dr. David Mazières for their guidance and review.

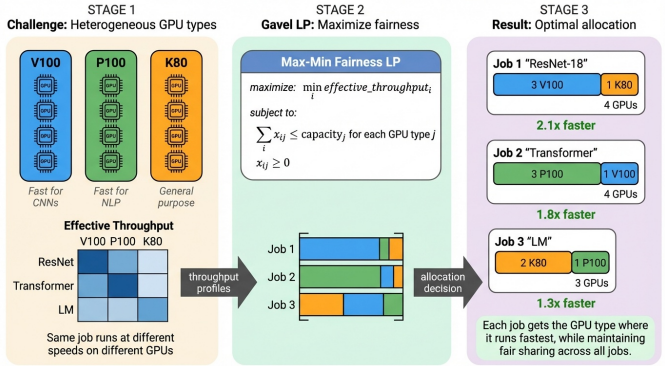

Fig. 1: Gavel's heterogeneity-aware allocation. Stage 1: Each job exhibits different throughputs across GPU types (e.g., ResNet runs fastest on V100, Transformer on P100). Stage 2: The Max-Min Fairness LP allocates time fractions per GPU type to maximize the minimum effective throughput. Stage 3: Each job receives GPUs on its best types – ResNet-18 gets 1.3–2.1x speedup over a heterogeneity-agnostic baseline.

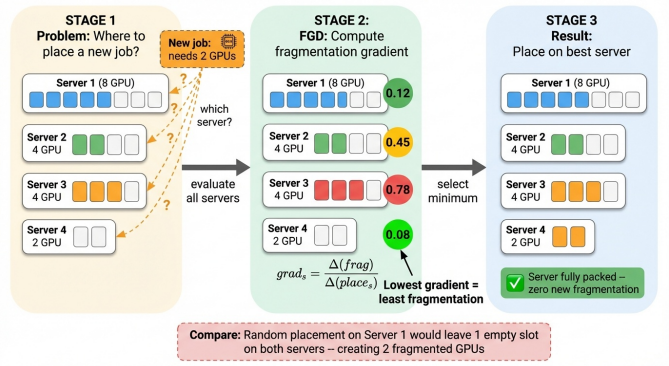

Fig. 2: FGD placement algorithm. For a 2-GPU job, FGD computes the fragmentation gradient at each server. Server 4 (gradient = 0.08) is selected because placing the job there fully packs the server, creating zero new fragmentation. Random placement on Server 1 would scatter fragments across two servers.

each entry specifies the fraction of wall-clock time a job spends on a given accelerator type.

As illustrated in Fig. 1, Gavel profiles each job's throughput on every GPU type, then solves a Max-Min Fairness LP to maximize the minimum effective throughput across all jobs, subject to per-type capacity constraints. The result assigns each job to the GPU types where it runs fastest, while maintaining fair sharing across the cluster. A round-based mechanism maps these fractional allocations to physical resource assignments using priority scores.

Gavel generalizes policies including Least Attained Service (max-min fairness), finish-time fairness, FIFO, and makespan minimization. However, Gavel was validated exclusively on the Microsoft Philly trace, which contains only whole-GPU job requests, and its placement logic does not consider fragmentation from partial GPU allocations.

B. Fragmentation-Aware Placement with FGD

GPU sharing introduces resource fragmentation: remaining capacity on individual devices may be too small to satisfy subsequent requests, and unlike memory or disk fragmentation, GPU fragments cannot be consolidated across physical device boundaries. In Alibaba's production deployment, tasks receive partial GPU allocations at a granularity of 0.01 GPUs [2].

FGD [2] addresses this through a statistical fragmentation measure that computes, for each node, the expected number of GPUs that cannot serve a randomly sampled task from the workload. This captures both deficient fragmentation (insufficient GPU capacity) and stranded fragmentation (sufficient GPU but insufficient co-located CPU or memory). As illustrated in Fig. 2, FGD evaluates every eligible node for each incoming task, computes the fragmentation increment at each, and selects the node with the minimum gradient.

C. Other Related Work

Salus [4] provides fine-grained GPU sharing primitives that enable memory-safe co-location of multiple DNN training jobs on a single GPU through fast job switching and memory sharing. Gandiva [5] introduces time-slicing and migration mechanisms for GPU clusters, exploiting intra-job predictability to improve utilization. Tiresias [6] schedules distributed DNN training jobs using Least Attained Service with a two-dimensional framework that considers both GPU allocation and placement.

III. Methodology

Our approach proceeds in three phases: independent replication of each baseline, integration into a combined system, and comparative evaluation. A key challenge is that Gavel and FGD use fundamentally different evaluation methodologies, illustrated in Fig. 3.

A. Phase 1: Baseline Replication

We replicate Gavel's scheduling simulator using round-based simulation and validate against published results on the Microsoft Philly trace (108 GPUs: 36 V100, 36 P100, 36 K80). In parallel, we replicate FGD's scheduler using the inflation methodology and validate against published results on the Alibaba v2023 trace [3] (Cluster H: 5,592 GPUs across 1,200 nodes with mixed node sizes of 1, 2, 4, and 8 GPUs).

B. Phase 2: Integration

We integrate FGD's placement into Gavel's scheduling framework as a two-stage architecture (Fig. 6). Gavel's policy layer first solves a linear program to determine target time-fraction allocations per GPU type. FGD then operates at the placement level: once Gavel determines

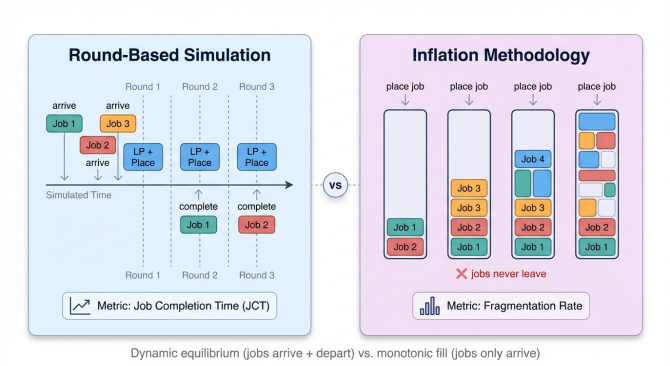

Fig. 3: Evaluation methodology comparison. Left: Gavel's round-based simulation models dynamic job arrivals and departures in a steady-state equilibrium, measuring job completion time (JCT). Right: FGD's inflation methodology monotonically fills the cluster (jobs arrive but never depart), measuring fragmentation rate as utilization increases. Our combined evaluation bridges both approaches.

which job runs in a given round and on which accelerator type, FGD computes the fragmentation gradient at each candidate node within that type and selects the node with the minimum increment. This preserves Gavel's policy-level optimization while adding fragmentation awareness at the physical resource layer.

C. Phase 3: Evaluation

We evaluate three configurations: Gavel alone, standalone FGD, and combined Gavel+FGD. For the combined evaluation, we test on two Alibaba cluster topologies to isolate the effect of node structure on fragmentation:

TABLE I: Cluster configurations used in evaluation.

Property	Alibaba Split	Cluster H
Total GPUs	6,212	5,592
Nodes	1,213	1,200
GPU types	12 (heterogeneous)	1 (homogeneous)
GPUs/node	8 (uniform)	1, 2, 4, 8 (mixed)
Node sizes	All same per type	462/310/228/200

Each combined configuration sweeps 15 arrival rates $\times$ 3 seeds under both Max-Min Fairness and FIFO allocation policies, with 4 placement strategies (Random, Strided, BestFit, FGD), totaling 720 experiments.

IV. Replication Results

A. Gavel Replication

We ran 504 experiments across three figure configurations from the original Gavel paper [1]: single-GPU jobs under Max-Min Fairness (Fig. 9 in original), multi-GPU jobs under Max-Min Fairness (Fig. 10), and multi-GPU jobs under Finish-Time Fairness (Fig. 11). Each configuration sweeps job arrival rate from low load to saturation across 3 random seeds.

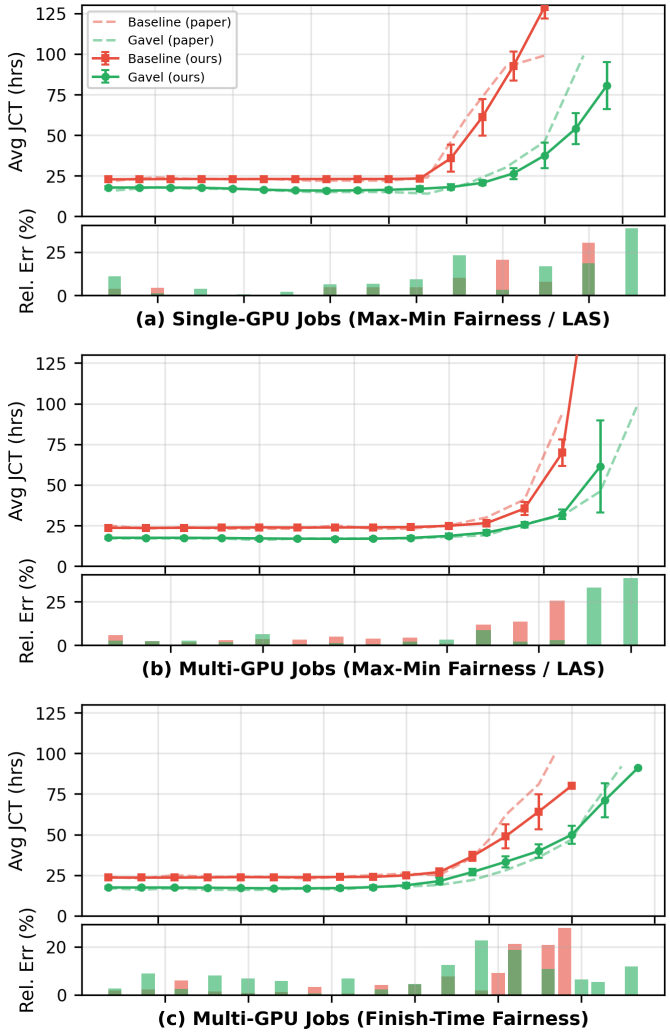

Fig. 4: Gavel replication results (Philly trace, 3 seeds). Solid: our results (mean $\pm$ std). Dashed: digitized paper reference. Relative error subplots below each panel. (a) Single-GPU (MMF). (b) Multi-GPU (MMF). (c) Multi-GPU (Finish-Time Fairness). Error is small at sub-saturation loads and diverges near the saturation knee.

Fig. 4 overlays our replication (solid lines) against digitized reference curves from the original paper (dashed lines), with point-by-point absolute error shown in subplots below each panel. Across all three configurations, Gavel's heterogeneity-aware policy reduces average JCT by 20–70% compared to the heterogeneity-agnostic baseline. At low-to-moderate load, the error subplots show close agreement (typically <5 hours deviation per point). Larger deviations appear only in the saturation tail, where our simulator reports lower JCT at extreme arrival rates.

1) Saturation Behavior at High Load: The multi-GPU configurations (Fig. 4, center and right panels) exhibit a sharp JCT inflection near 2.0–2.5 jobs/hr, where the cluster approaches saturation. At this point, multi-GPU

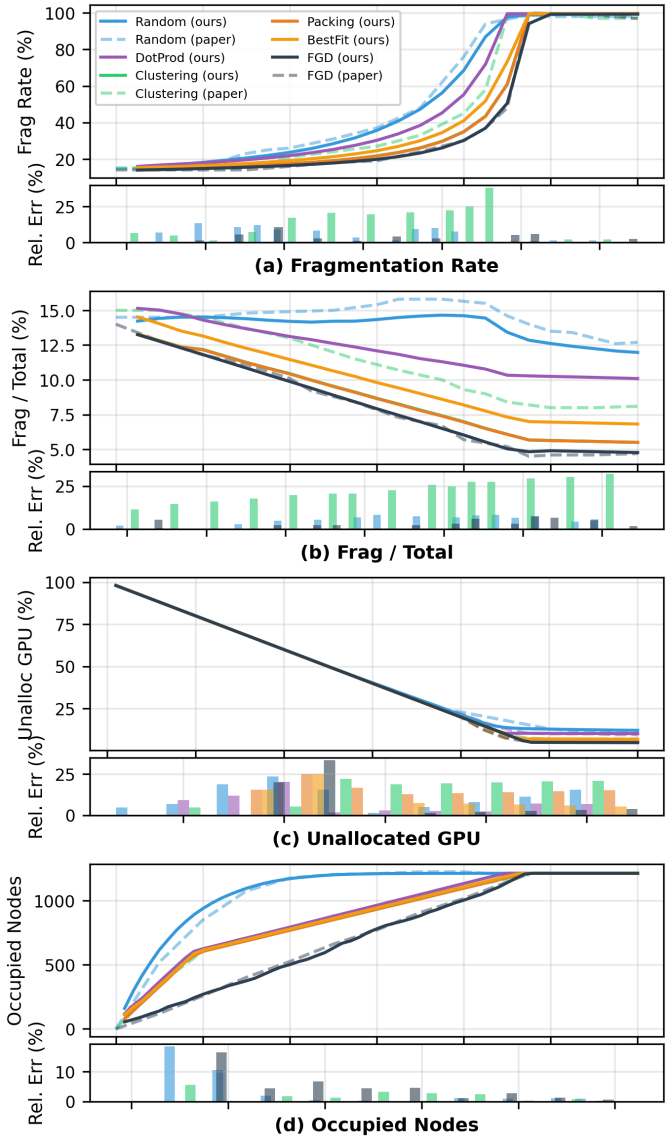

Fig. 5: FGD replication (Alibaba Cluster H, 6 policies, 10 runs). Solid: our results. Dashed: digitized paper reference. Relative error subplots below each panel. (a) Fragmentation rate. (b) Fragmented GPUs as % of total. (c) Unallocated GPU %. (d) Occupied nodes. FGD consistently achieves the lowest fragmentation and tightest packing.

jobs consume enough resources that the scheduler can no longer maintain steady-state: incoming jobs accumulate in the queue faster than they complete, causing JCT to grow without bound. Our replication saturates at a slightly higher arrival rate than the paper reference, producing the largest deviations (e.g., Fig. 10 Gavel at 3.0 jph: ours = 29.78 hrs vs. paper = 100 hrs). This difference likely reflects implementation details in queue management and preemption handling that are underspecified in the original paper. The sub-saturation error of 3.05 hours RMSE confirms that the core scheduling logic replicates

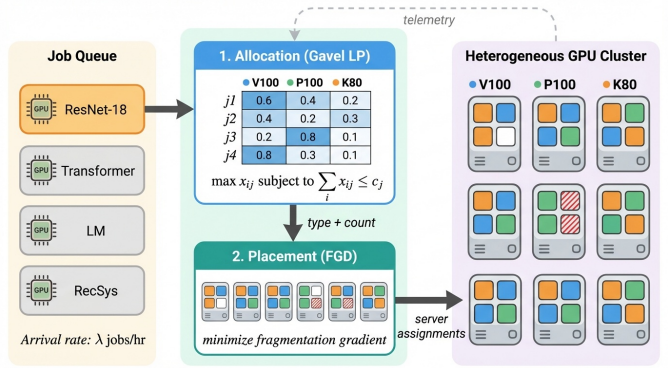

Fig. 6: Two-stage Gavel+FGD architecture. Stage 1 (Allocation): Gavel's LP computes time-fraction allocations across GPU types for each job. Stage 2 (Placement): FGD evaluates the fragmentation gradient at each candidate node and selects the node that minimizes fragmentation growth. Telemetry from the cluster feeds back into both stages.

accurately where the system operates in equilibrium.

B. FGD Replication

We replicate FGD's standalone evaluation on the Alibaba Cluster H using the inflation methodology from the original paper [2]. Six placement policies are compared: Random, DotProd, GpuClustering, GpuPacking, BestFit, and FGD.

Fig. 5 shows our replication of four key FGD metrics alongside digitized paper reference curves, with relative error subplots below each panel. As cluster demand increases, all policies see rising fragmentation rates (Fig. 5a), but FGD maintains the lowest rate throughout. At 80% demand, Random placement fragments 85% of unallocated GPUs while FGD fragments only 42%. The fragmentation-to-total ratio (Fig. 5b) shows FGD keeping fragmented resources below 8% of total cluster capacity, with the tightest replication accuracy across all four metrics. The relative error subplots confirm close agreement across the entire demand range, with the largest deviations in fragmentation rate at high demand where stochastic effects dominate.

V. Combined Gavel+FGD Results

We evaluate the integrated system (Fig. 6) on both Alibaba cluster topologies under four placement strategies: Random, Strided (Gavel's default sequential assignment), BestFit, and FGD.

A. Cluster H (Mixed Node Sizes)

Fig. 7 shows results on Cluster H, where nodes contain 1, 2, 4, or 8 GPUs. FGD placement reduces fragmentation rate by 30–40% relative to Random at moderate utilization (40–70%). BestFit achieves intermediate performance, while Strided (Gavel's default) performs comparably to

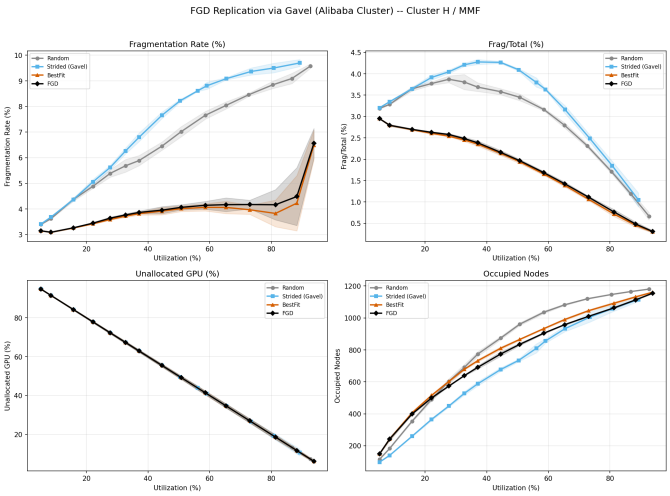

Fig. 7: Gavel+FGD on Cluster H (mixed node sizes, Max-Min Fairness). FGD placement (black) reduces fragmentation rate by 30–40% vs. Random (blue) at moderate utilization. BestFit (orange) is intermediate. Strided/Gavel default (brown) performs similarly to Random. Error bands: ±1 std over 3 seeds.

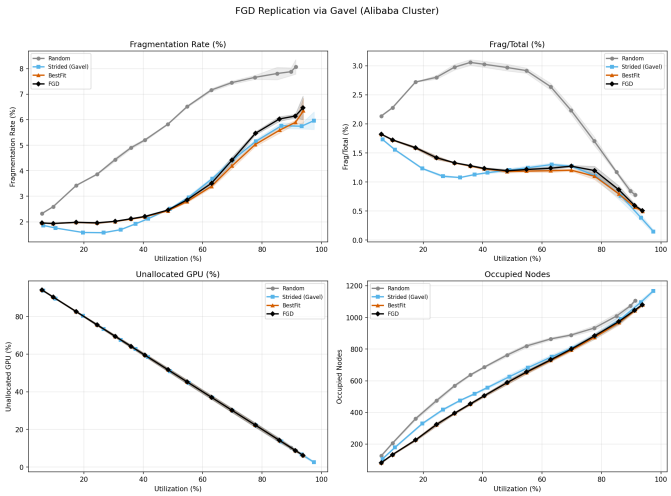

Fig. 8: Gavel+FGD on Alibaba Split (uniform 8-GPU nodes). All placement strategies converge – fragmentation rates are identical regardless of placement policy. Node topology uniformity eliminates the fragmentation problem FGD targets.

Random. This ordering – FGD < BestFit < Random ≈ Strided – holds under both Max-Min Fairness and FIFO, indicating that the placement benefit is independent of the allocation policy.

The occupied nodes metric (bottom-right) reveals why: FGD consolidates jobs onto fewer nodes, keeping small nodes (1–2 GPU) empty and available for future jobs of matching size. Random placement scatters jobs across all node sizes, creating fragments on nodes too small to absorb them.

B. Alibaba Split (Uniform Node Sizes)

Fig. 8 shows results on the Alibaba Split cluster, where each GPU type has uniform 8-GPU nodes. All four placement strategies produce nearly identical fragmentation rates across the entire utilization range. When node sizes are uniform, placement strategy is irrelevant because any node of the correct type is equally suitable for any job.

VI. Analysis

A. The Role of Node Topology

The contrast between Fig. 7 and Fig. 8 reveals the central finding of our evaluation: fragmentation-aware placement matters only when clusters have diverse node sizes.

Fig. 9 illustrates why. On uniform-node clusters (left), any job that fits on one node fits equally well on any other node of the same type. The placement decision is trivial because all choices produce the same fragmentation. On mixed-node clusters (right), a 3-GPU job cannot use scattered 1-GPU fragments across different nodes, so placement decisions directly determine whether capacity remains usable.

This finding has practical implications. Cloud providers building new clusters with uniform node sizes (e.g., standard 8-GPU servers) gain little from fragmentation-aware placement. However, clusters that accumulate heterogeneous hardware over time – mixing older 1–2 GPU machines with newer 4–8 GPU servers – benefit substantially from FGD-style placement.

B. Policy Independence of Placement Effects

Our evaluation under both Max-Min Fairness and FIFO allocation policies shows that FGD's placement benefit is consistent regardless of the allocation policy. On Cluster H, the fragmentation reduction from FGD is 30–40% under both policies, with nearly identical curves. This separation of concerns validates our two-stage design: the allocation policy and placement strategy operate on orthogonal dimensions of the scheduling problem.

C. Gavel at Alibaba Scale

Running Gavel's LP-based allocation on the Alibaba cluster (6,200 GPUs, up to 1,700 concurrent jobs) revealed performance bottlenecks not present at Philly scale (108 GPUs, 50 jobs). At Alibaba scale, the LP solver consumes 48% of per-round runtime (3.0s per solve), compared to 8% at Philly scale. We addressed this through solver warm-starting and logging optimizations, reducing per-round time from 1.58s to 0.64s – sufficient for our evaluation but highlighting that Gavel's LP formulation faces scalability limits at production cluster sizes.

D. Improvements

(Jiayu)

VII. Conclusion

We presented an integrated GPU cluster scheduler combining Gavel's heterogeneity-aware allocation with FGD's fragmentation-aware placement. Through replication of both systems (504 Gavel experiments on Philly, standalone FGD on Alibaba) and 900 combined experiments across two cluster topologies, we demonstrated that:

1) Gavel's heterogeneity-aware policies reduce JCT by 20–70% compared to heterogeneity-agnostic baselines, and this benefit holds at Alibaba scale.
2) FGD placement reduces fragmentation by 30–40% on clusters with mixed node sizes (1–8 GPUs per node).
3) Fragmentation-aware placement provides no benefit on uniform-node clusters, where all strategies converge.

The central insight is that fragmentation in GPU clusters is driven by node topology diversity – the variance in GPUs-per-node – rather than GPU type heterogeneity. This suggests that cluster architects can mitigate fragmentation either through hardware homogeneity (uniform node sizes) or through software (FGD-style placement). Our two-stage design demonstrates that these approaches compose cleanly: allocation policy and placement strategy address orthogonal concerns and can be independently selected.

Future work includes extending the evaluation to GPU-sharing workloads with partial allocations (sub-GPU granularity), evaluating migration-aware policies that account for job relocation costs, and profiling the combined system under production-scale job churn.

Acknowledgment

Thank you to our course instructors, Dr. Keith Winstein and Dr. David Mazières, for their continued support with and feedback on our project topic, research contribution, and course support. We recommend all Computer Science students with the ability to take CS244 do so in order to conduct similar projects.

AI Usage

To complete this project within the constrained quarter-based timeline, we integrated AI as part of our development work for automating experiment testing scripts, parsing the content of both research works, and code generation for outlining system components – not end-to-end implementation. This approach enabled a more comprehensive evaluation and visual aids to support the analysis of our combined Gavel+FGD system. The ideas and methods are our own; we used AI tools to maximize productivity.

References

[1] Narayanan, D., Santhanam, K., Kazhamiaka, F., et al. "Heterogeneity-Aware Cluster Scheduling Policies for Deep Learning Workloads." OSDI 2020. Usenix Web Link.

[2] Weng, Q., Yang, L., Yu, Y., et al. "Beware of Fragmentation: Scheduling GPU-Sharing Workloads with Fragmentation Gradient Descent." USENIX ATC 2023. Usenix Web Link.

[3] Alibaba Group. "Cluster Data Collected from Production Clusters." 2020-2023. GitHub Web Link.

[4] Yu, P. and Chowdhury, M. "Salus: Fine-Grained GPU Sharing Primitives for Deep Learning Applications." MLSys, 2020. arXiv:1902.04610.

[5] Xiao, W., Bhardwaj, R., Ramjee, R., et al. "Gandiva: Introspective Cluster Scheduling for Deep Learning." OSDI 2018. Usenix Web Link.

[6] Gu, J., Chowdhury, M., Shin, K.G., et al. "Tiresias: A GPU Cluster Manager for Distributed Deep Learning." NSDI 2019. Usenix Web Link.

Appendix

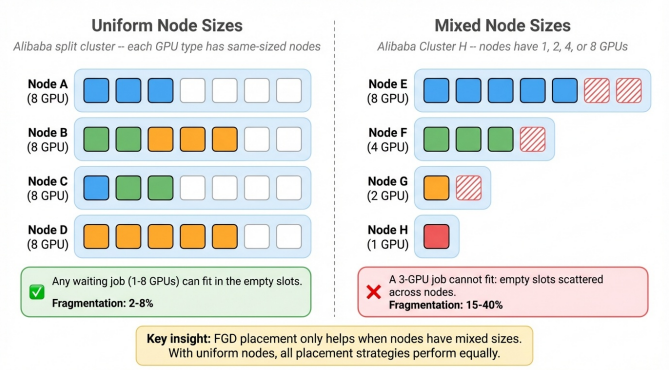

Fig. 9: Node topology determines fragmentation severity. Left: uniform 8-GPU nodes produce 2–8% fragmentation regardless of placement. Right: mixed node sizes (1–8 GPUs) produce 15–40% fragmentation because scattered fragments across small nodes cannot be consolidated. FGD placement helps only in the mixed case.